

Interrupts und Timer

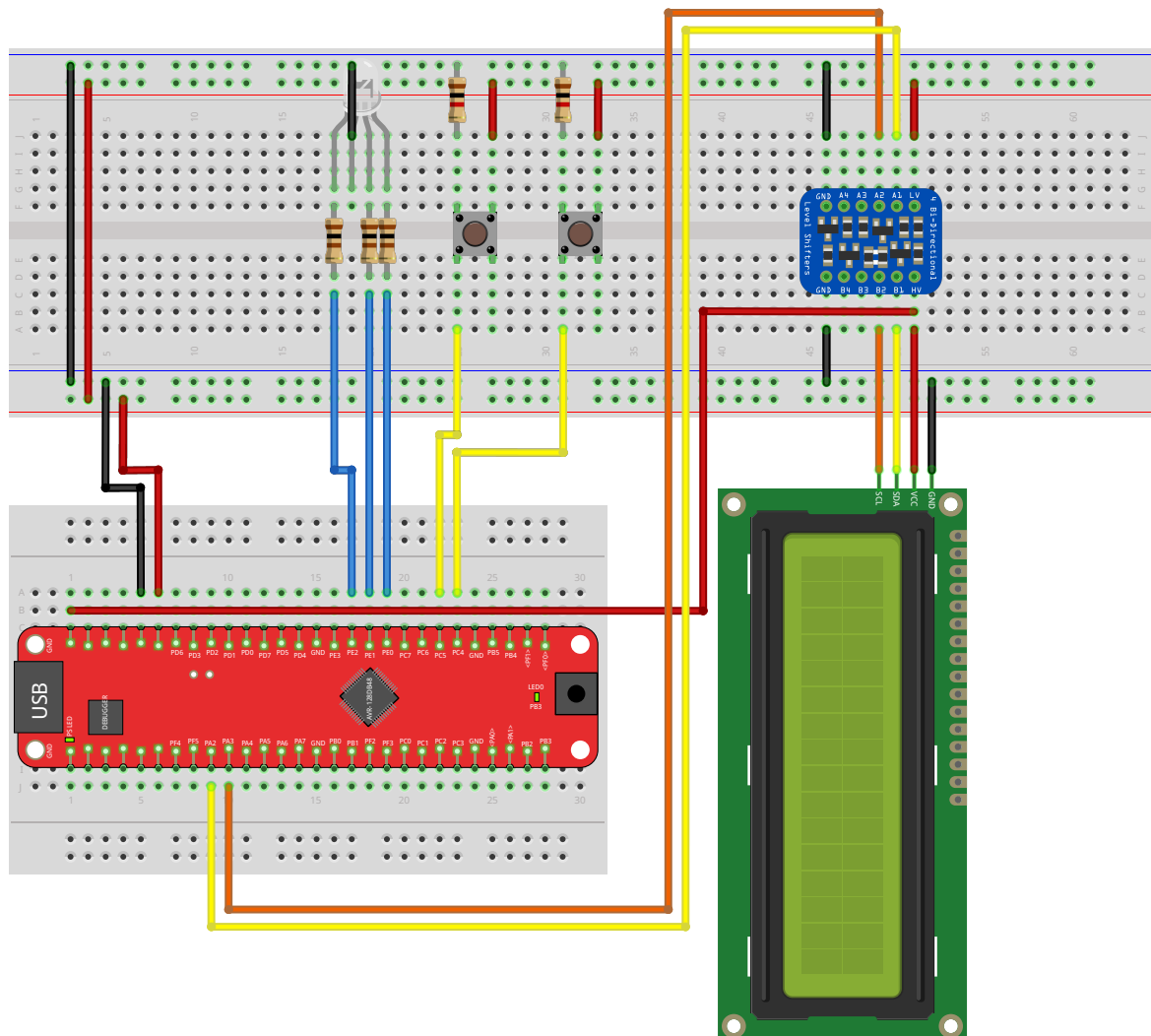
Übersicht

Aufgaben	2
Button Interrupt	3
Timer	3
Traffic Light	3
Programmable Timer	4
Literaturverzeichnis	5

Aufgaben

In dieser Übung verändern wir nun wieder unseren Versuchsaufbau, entfernen zwei der vier Buttons und nutzen, wie in [Versuchsaufbau 4](#) zu erkennen, die Pins 0, 1 & 2 an Port E, um eine RGB-LED anzuschließen. Die Anzeige des LCDs und die RGB-LED sollen in den folgenden Aufgaben mit Hilfe der internen Interrupts und Timer angesteuert werden.

Inkludieren Sie dazu in Ihren Programmen die Bibliothek *avr/interrupt.h*.



fritzing

Abbildung 1: Ein Liquid Crystal Display, zwei Buttons und eine RGB-LED. Das TXB0104-Bauteil stammt aus der Adafruit Fritzing Library[1]

Wichtiger HINWEIS

Für alle Aufgaben in dieser Veranstaltung ist lediglich die TCA Peripherie nötig. Verwenden Sie andere Bausteine **nur**, wenn Sie dafür einen guten Grund nennen können. **Die Verwendung anderer Timerbausteine führt zu Punkteabzug.**

Desweiteren verwenden Sie ab diesem Kapitel (inklusive) **auf keinen Fall** mehr aktives Warten (blocking delay), außer es ist ausdrücklich erlaubt.

Button Interrupt

Schalten Sie die RGB-LED durch eine Interrupt-Service-Routine (ISR) in einer beliebigen Farbe an, sobald der Button an Pin C4 gedrückt wird. Sobald der Button noch einmal gedrückt wird, soll die LED wieder ausgeschaltet werden.

Erweiterung: Bei jedem weiteren Einschalten soll die LED mit einer neuen, zufälligen Farbe (auch Kombinationen der Grundfarben) eingeschaltet werden. Nutzen Sie **keine** bereits vorhandenen Bibliotheken, sondern implementieren Sie den Xorshift32-Algorithmus als Pseudozufallszahlengenerator. Definieren Sie dessen Anfangszustand, um eine einfache Änderung innerhalb Ihres Codes zu ermöglichen.

- Was versteht man unter einem Pseudozufallszahlengenerator?

Timer

Schreiben Sie ein Programm, welches auf dem LCD einen Sekundenzähler ausgibt. Die LCD-Anzeige soll sich dabei **exakt** jede vergangene Sekunde aktualisieren und den aktuellen Zählerstand anzeigen. Nutzen Sie dazu Timer und deren Interruptfunktionalität. Beachten Sie die Richtlinien der Interrupt-Service-Routinen:

- Reduzieren Sie deren Komplexität und Laufzeit so weit wie möglich.
- Seien Sie sich bewusst, wann welche Flags gesetzt und gelöscht werden müssen.

Traffic Light

Schreiben Sie ein Programm, welches eine einfache Ampelschaltung simuliert. Lassen Sie die RGB-LED auf Grün wechseln, wenn der Button an Pin C4 gedrückt wurde und die Ampel bereits Rot ist und lassen Sie die Ampel auf Rot wechseln, wenn der Button an Pin C5 gedrückt wurde und die Ampel bereits Grün ist.

Denken Sie daran, dass Ampeln Gelbphasen haben und zeitverzögert arbeiten!

Die zeitlichen Übergänge sollen mittels Timer-Interrupts realisiert werden, die Knopfdrücke sollen ebenfalls mittels ISR aufgefangen werden.

- Macht es Sinn, während einer wechselnden Ampelphase auf einen Knopfdruck zu reagieren?
- Angenommen, der Knopf sowie der Timer würden **zeitgleich** ihre ISR auslösen. Führt dies zu undefiniertem Verhalten?

Programmable Timer

In [der vorherigen Aufgabe](#) haben Sie einen Sekundenzähler implementiert und diesen auf dem LCD wiedergegeben. Schreiben Sie nun mithilfe von Interrupts einen über die beiden Buttons programmierbaren Timer, welcher nach **abgelaufener** Zeit die RGB-LED rot aufleuchten lässt.

Nutzen Sie einen der beiden Buttons, um 5 Sekunden auf die ablaufende Zeit zu addieren und den anderen Button, um den Timer starten und pausieren zu lassen. Die aktuell übrige Sekundenanzahl soll jederzeit auf dem LCD abzulesen sein.

Schreiben Sie passende Interrupts-Service-Routinen, die abgehandelt werden, sobald

- der Timer gestartet oder pausiert werden soll,
- Zeit zum Timer hinzugefügt werden soll und
- der Timer abgelaufen ist.

Literaturverzeichnis

- [1] Adafruit Industries, „Adafruit Fritzing Library“. [Online]. Verfügbar unter: <https://github.com/adafruit/Fritzing-Library>